

UVM

Universal Verification Methodology

A Practical APB-UART Verification Case Study

Architecture · Phases · Objections · Drivers · Monitors · Scoreboards · Coverage · Extension

[6 Chapters](#) · [UVM Vocabulary](#) · [Reference Tables](#) · [Real-World Testbench](#)

© 2026 squared-studio · squared-studio.cc/SkillVerse

uvm

1. Chapter 1

- What You Should Learn in This Chapter
- 1.1 Why This Example Is Good for a Trainee Engineer
- 1.2 What APB Means in This Tutorial
- 1.3 What UART Means in This Tutorial
- 1.4 The Main Registers You Need to Care About
- 1.5 The Core Verification Question
- 1.6 UVM Vocabulary in Plain Language
- 1.7 Why Transactions Are Better Than Raw Signal Thinking
- 1.8 The Main Learning Goal Before Chapter 2
- Next Chapter

2. Chapter 2

- What You Should Learn in This Chapter
- 2.1 The Big Picture of the Environment
- 2.2 Why the Loopback Setup Is So Helpful
- 2.3 What the Top-Level Module Should Do
- 2.4 What Is a Virtual Interface?
- 2.5 Why uvm_config_db Exists
- 2.6 What the Environment Owns
- 2.7 What the Architecture Diagram Is Telling You
- 2.8 The Main Lesson of This Chapter
- Previous and Next

3. Chapter 3

- What You Should Learn in This Chapter
- 3.1 Why Phases Exist at All
- 3.2 The Most Important Phases in This Example
- 3.3 Why the Base Test Is So Important
- 3.4 A Close Look at the Configuration Phase
- 3.5 What an Objection Really Means
- 3.6 Why Derived Tests Should Stay Small
- 3.7 The Role of Reset in Verification Quality
- 3.8 The Main Lesson of This Chapter
- Previous and Next

4. Chapter 4

- What You Should Learn in This Chapter
- 4.1 What Is Inside an Agent?
- 4.2 Why APB and UART Use Different Agents
- 4.3 The Basic Write Test
- 4.4 The Basic Read Test

- [4.5 Why Waiting for Idle Matters](#)
- [4.6 What You Should Be Able to Explain Now](#)
- [Previous and Next](#)

5. Chapter 5

- [What You Should Learn in This Chapter](#)
- [5.1 Why the Scoreboard Is So Important](#)
- [5.2 The Scoreboard's Three Main Jobs](#)
- [5.3 A Simplified View of the Compare Logic](#)
- [5.4 What the Scoreboard Is Really Proving](#)
- [5.5 Why Coverage Still Matters](#)
- [5.6 What Each Core Test Proves](#)
- [5.7 What the Scoreboard Does Not Automatically Prove](#)
- [5.8 The Main Lesson of This Chapter](#)
- [Previous and Next](#)

6. Chapter 6

- [What You Should Learn in This Chapter](#)
- [6.1 A Safe Way to Think About Extensions](#)
- [6.2 Reuse Before You Create](#)
- [6.3 A Minimal Derived Test Template](#)
- [6.4 When the Scoreboard Needs to Grow](#)
- [6.5 When Coverage Needs to Grow](#)
- [6.6 Common Beginner Mistakes](#)
- [6.7 A Practical Checklist for Writing a New Test](#)
- [6.8 Final Takeaway for a New Verification Engineer](#)
- [Previous](#)
- [Back to Index](#)

Chapter 1: Understanding the Problem and the Language

What You Should Learn in This Chapter

By the end of this chapter, you should be comfortable with three things:

- what the APB-UART block is supposed to do,
- why it is a good beginner example for UVM,
- and what the most common UVM words mean in plain language.

If you are completely new to UVM, start here and do not rush. A lot of confusion later comes from not having a clear picture of the DUT and the vocabulary first.

1.1 Why This Example Is Good for a Trainee Engineer

UVM is easier to learn when the design under test is small enough that you can follow one transaction all the way through the verification flow.

An APB-UART design is useful because it naturally splits into two views:

- a software-facing APB register view,
- a serial-data UART view.

That means you can ask very practical verification questions:

- If software writes a byte into the transmit register, does that byte really leave the UART transmitter?
- If a serial byte arrives on the receive pin, does software later read the same byte back through APB?

These are strong beginner questions because they are both realistic and easy to visualize.

1.2 What APB Means in This Tutorial

APB, or Advanced Peripheral Bus, is a simple on-chip bus protocol commonly used for peripheral control registers.

In this tutorial, you can think of APB as the path used by software to:

- enable the UART,

```
program the baud-rate divider,
```

- configure parity and stop bits,
- write outgoing data,
- read incoming data.

So when you see APB traffic in this environment, think, "software is controlling the block through registers."

1.3 What UART Means in This Tutorial

UART, or Universal Asynchronous Receiver/Transmitter, is a serial communication interface.

Unlike APB, UART does not move data as a wide parallel bus transaction. It sends data one bit at a time using a frame that typically contains:

- a start bit,
- data bits,
- an optional parity bit,
- one or more stop bits.

In this tutorial, you can think of the UART side as the physical communication behavior of the DUT.

So when you see UART monitor traffic, think, "this is what actually happened on the serial line."

1.4 The Main Registers You Need to Care About

You do not need the full register map memorized to understand the training flow. A few addresses are enough.

Address	Register	Why it matters
0x00	CTRL	Turns the UART on and controls flush behavior
0x04	CLK_DIV	Sets the baud-rate divider
0x08	CFG	Controls parity and stop-bit format
0x14	TX_DATA	APB writes a byte here for transmission
0x18	RX_DATA	APB reads a received byte here

If you remember only two addresses, remember these:

- 0x14 is the APB transmit data write point,
- 0x18 is the APB receive data read point.

These two registers are the center of the scoreboard's end-to-end data checks.

1.5 The Core Verification Question

A beginner sometimes thinks verification means, "send some stimulus and hope nothing crashes."

That is not enough.

A better way to think about verification is this:

- define what behavior should happen,
- create stimulus that exercises that behavior,
- observe what really happened,
- compare expected behavior and observed behavior.

For this APB-UART example, the core verification questions are:

- When APB writes TX_DATA, does the UART transmit side emit the same byte?
- When the UART receive side gets a byte, does APB later read the same byte from RX_DATA?

Those two questions are simple, strong, and easy to measure.

1.6 UVM Vocabulary in Plain Language

Before moving further, it helps to translate the main UVM terms into simple engineering language.

UVM term	Plain meaning	Why it exists
DUT	The hardware block being verified	We need a short way to refer to the design under test
transaction	One meaningful operation such as an APB read, APB write, or UART byte event	Easier to reason about than raw signals
sequence item	The object that holds one transaction	Carries address, data, direction, and control information
sequence	A generator of one or more sequence items	Describes what stimulus we want
sequencer	The coordinator between sequence and driver	Hands transactions to the driver
driver	The component that converts a transaction into interface signal activity	Talks to the DUT
monitor	The component that observes interface signals and reconstructs transactions	Reports what really happened
agent	The container for sequencer, driver, and monitor of one interface	Organizes one protocol's verification logic
environment	The container that gathers multiple agents and checkers	Builds the full verification system
scoreboard	The checker that compares results	Detects mismatches
functional coverage	A measurement of which important situations were exercised	Helps answer what has been tested

A simple trainee-friendly memory trick is:

- sequences decide what we want to do,
- drivers make it happen on signals,
- monitors tell us what actually happened,
- scoreboards decide whether it was correct.

1.7 Why Transactions Are Better Than Raw Signal Thinking

When beginners first see UVM, they often wonder why everything is wrapped in classes and objects instead of just working with pins directly.

The answer is abstraction.

Consider an APB write. At the signal level, you have to think about:

- psel,
- penable,
- paddr,
- pwrite,
- pwwdata,
- pstrb,
- pready.

That is correct, but it is too low level to think clearly about test intent all the time.

At the transaction level, the same action becomes something much simpler:

```
direction = write,
```

```
address = 0x14,
```

```
data = one byte.
```

UVM lets the test describe intent at the transaction level while the driver handles the signal-level protocol details.

That is one of the biggest reasons UVM scales better than handwritten, signal-level procedural benches.

1.8 The Main Learning Goal Before Chapter 2

If you leave this chapter with a clear answer to the question below, you are ready for the next chapter:

"Can I explain, in simple words, why this DUT has two different observable views and why a scoreboard can compare them?"

If yes, continue.

If not, go back to the APB and UART descriptions and the two main verification questions. Those are the foundation for everything that comes next.

Chapter 2: Testbench Architecture and Top-Level Flow

What You Should Learn in This Chapter

This chapter explains how the overall UVM testbench is organized.

By the end, you should understand:

- how the major blocks connect together,
- why loopback makes this tutorial easier to follow,
- what the top-level module is responsible for,
- and why virtual interfaces are necessary.

2.1 The Big Picture of the Environment

The example environment is intentionally small. That is a strength, not a limitation.

It contains:

- one top-level testbench module,
- one base test,
- one environment,
- one APB agent,
- one UART agent,
- one scoreboard,
- and a small number of focused tests and sequences.

A good high-level picture of the verification data flow is:

+-----+

UVM test

+-----+

V

+-----+

sequences

+-----+

+-----+

V

V

+-----+ +-----+

APB sequencer

UART sequencer


```

+-----+-----+      +-----+-----+
V           V
+-----+-----+      +-----+-----+

```

APB driver

UART driver

```

+-----+-----+      +-----+-----+
+-----+-----+
V V
+-----+

```

DUT

```

+-----+
+-----+ +-----+
V           V
+-----+-----+      +-----+-----+

```

APB monitor

UART monitor

```

+-----+-----+      +-----+-----+
+-----+-----+
V V
+-----+

```

scoreboard

```

+-----+

```

Read that diagram slowly.

- Tests and sequences create intent.
- Drivers turn that intent into real interface activity.
- Monitors observe what actually happened.
- The scoreboard compares observations and decides whether the DUT behaved correctly.

That is the heart of the UVM model.

2.2 Why the Loopback Setup Is So Helpful

A beginner-friendly feature of this APB-UART environment is the loopback-style connection between the UART pins.

A representative DUT hookup looks like this:

```
uart_top #(
  .ADDR_WIDTH(SYS_ADDR_WIDTH),
  .DATA_WIDTH(SYS_DATA_WIDTH)
) udut (
  .arst_ni(ctrl_intf.arst_ni),
  .clk_i(ctrl_intf.clk_i),
  .psel_i(apb_intf.psel),
  .penable_i(apb_intf.penable),
  .paddr_i(apb_intf.paddr),
  .pwrite_i(apb_intf.pwrite),
  .pwrdata_i(apb_intf.pwrdata),
  .pstrb_i(apb_intf.pstrb),
  .pready_o(apb_intf.pready),
  .prdata_o(apb_intf.prdata),
  .pslverr_o(apb_intf.pslverr),
  .rx_i(uart_intf.tx),
  .tx_o(uart_intf.rx)
);
```

In plain language, the transmit and receive sides are cross-connected so the environment can observe complete paths without needing a second external UART device model.

That gives us clean end-to-end stories:

- write a byte through APB,
- watch it emerge through the UART path,
- inject UART receive traffic,
- read the same data back through APB.

For a trainee engineer, this removes a lot of distraction.

2.3 What the Top-Level Module Should Do

A common beginner mistake is to put too much verification logic in the top-level module. In UVM, the top-level module should stay thin.

Its main responsibilities are:

- instantiate the DUT,
- instantiate interfaces,
- publish those interfaces and parameters to the class world,
- start the UVM test.

A representative startup block looks like this:

```
initial begin
    string test_name;

    if (!$value$plusargs("UVM_TESTNAME=%s", test_name) &&
```

```
!$value$plusargs("test=%s", test_name)) begin
```

```
    test_name = "base_test";
end

    uvm_config_db#(int)::set(uvm_root::get(), "parameter", "ADDR_WIDTH",
ADDR_WIDTH);

    uvm_config_db#(int)::set(uvm_root::get(), "parameter", "DATA_WIDTH",
DATA_WIDTH);

    uvm_config_db#(virtual ctrl_if)::set(uvm_root::get(), "ctrl", "ctrl_intf",
ctrl_intf);

    uvm_config_db#(virtual uart_if)::set(uvm_root::get(), "uart", "uart_intf",
uart_intf);

    uvm_config_db#(virtual apb_if)::set(uvm_root::get(), "apb", "apb_intf",
apb_intf);
```

```
run_test(test_name);
```

```
end
```

A trainee engineer should learn three important ideas from this block.

Runtime test selection

The same compiled testbench can run different tests depending on the plusarg. This is one reason UVM scales well.

Configuration publishing

The module does not manually hand values to every class instance. Instead, it publishes information through `uvm_config_db`.

Thin top-level design

The top-level module bootstraps the UVM class world. It should not become a second procedural testbench that competes with the class hierarchy.

2.4 What Is a Virtual Interface?

This is one of the most common beginner questions.

SystemVerilog modules can instantiate real interfaces directly. SystemVerilog classes cannot interact with those interfaces the same way unless they receive a handle.

A virtual interface is that handle.

The flow is:

- the top-level module creates real interfaces,
- the module stores virtual-interface handles in `uvm_config_db`,
- drivers, monitors, and tests retrieve those handles and use them.

Without a virtual interface, a driver would not have a clean way to access APB or UART signals and tasks.

2.5 Why `uvm_config_db` Exists

At first glance, `uvm_config_db` can look like unnecessary complexity. But it solves a real organizational problem.

Many UVM components need shared information such as:

`interface handles,`

`parameter values,`

- default configuration values,

`sequence lengths,`

- protocol settings.

If everything were passed manually through constructors, the code would quickly become deeply coupled and hard to maintain.

`uvm_config_db` gives the environment a standard place to publish and retrieve configuration state.

A trainee engineer does not need to master every detail of it immediately. At this stage, it is enough to understand:

- the top level publishes things,
- components retrieve things,

- and this allows the class world to stay flexible.

2.6 What the Environment Owns

The environment is the top-level UVM class container for the main verification blocks.

A representative implementation looks like this:

```
class apb_uart_env extends uvm_env;
```

```
  apb_agent apb;
  uart_agent uart;
  apb_uart_scdb scdb;
```

```
  virtual function void build_phase(uvm_phase phase);
```

```
    super.build_phase(phase);
```

```
        apb  = apb_agent::type_id::create("apb", this);
        uart = uart_agent::type_id::create("uart", this);
        scdb = apb_uart_scdb::type_id::create("scdb", this);

    endfunction
```

```
    function void connect_phase(uvm_phase phase);
```

```
        super.connect_phase(phase);
        apb.mon.ap.connect(scdb.m_analysis_imp_apb);
        uart.mon.ap.connect(scdb.m_analysis_imp_uart);
```

```
    endfunction
```

```
endclass
```

This is a good beginner example because it shows what an environment should do:

- create its main subcomponents,
- connect them,
- avoid containing scenario-specific stimulus.

The environment is the structure. The tests provide the behavior.

2.7 What the Architecture Diagram Is Telling You

Do not treat the architecture figure as decoration. It gives you a visual map of the DUT's major blocks and data flow.

When you see a diagram like this in a verification tutorial, use it to ask:

- Which blocks are configured from APB registers?
- Which blocks influence transmission?
- Which blocks influence reception?
- Where would you expect status and data to be visible?

That habit will make both design debugging and verification planning easier later in your career.

2.8 The Main Lesson of This Chapter

If you leave this chapter understanding the following sentence, you are on track:

"The top-level module only boots the UVM world, the environment organizes the main blocks, and virtual interfaces allow class-based components to touch real interface signals."

That is the structural foundation for the rest of the tutorial.

Chapter 3: UVM Phases, Objections, and the Base Test

What You Should Learn in This Chapter

This chapter explains how the simulation moves through time at the UVM testbench level.

By the end, you should understand:

- what the main UVM phases are doing in this example,
- why the base test is so important,
- and what objections mean in practice.

3.1 Why Phases Exist at All

A beginner often sees UVM phases as an arbitrary list of method names. That makes them hard to remember.

A better way to think about phases is this:

Phases are a structured way to separate different kinds of work so the testbench does not become chaotic.

Instead of doing everything in one place, UVM encourages an ordered flow:

- build the components,
- connect the components,
- reset the DUT,
- configure the DUT,
- run the scenario,
- summarize the result.

That is much easier to maintain than an unstructured procedural test.

3.2 The Most Important Phases in This Example

This example uses a subset of the full UVM phase system, and that is a good thing for training.

build_phase

This is where components are created.

Typical actions here include:

- creating the environment,
- creating the APB and UART agents,
- creating the scoreboard.

connect_phase

This is where connections are made.

Typical actions here include:

- connecting monitor analysis ports to the scoreboard,
- retrieving virtual interfaces if the component needs them.

reset_phase

This is where the DUT is placed into a known, clean starting state.

Typical actions here include:

- asserting reset,
- clearing interface signals,
- waiting for the reset sequence to complete safely.

configure_phase

This is where the DUT is programmed for the upcoming test scenario.

Typical actions here include:

- setting baud-rate related values,
- programming parity and stop-bit settings,
- enabling the clock,
- sending an initial APB setup sequence.

main_phase

This is where the actual scenario stimulus runs.

Typical actions here include:

- randomized APB writes to transmit data,
- UART receive traffic generation,
- APB reads from receive data,
- register sweeps.

report_phase

This is where final summaries are produced.

Typical actions here include:

- reporting coverage information,
- reporting pass or fail status.

3.3 Why the Base Test Is So Important

The base test is one of the most important ideas in a reusable UVM environment.

Without a base test, every scenario test tends to repeat the same work:

- retrieve interfaces,
- reset the DUT,
- start the clock,

program default register values,

- enable the design.

That repetition causes two problems:

- duplicated code,
- inconsistent setup between tests.

A base test solves this by holding the setup that every test should share.

In this APB-UART example, the base test is responsible for:

- building the environment,

- retrieving virtual interfaces,
- applying reset,
- setting default UART parameters,
- enabling the clock,
- starting the initial APB configuration sequence.

That means derived tests can focus on the scenario they are trying to prove.

3.4 A Close Look at the Configuration Phase

The configuration phase is especially useful for training because it shows how setup is performed in a deliberate, readable way.

```
virtual task configure_phase(uvm_phase phase);
```

```
super.configure_phase(phase);
phase.raise_objection(this);
uart_intf.BAUD_RATE = 6_250_000;
uart_intf.PARITY_ENABLE = 0;
uart_intf.PARITY_TYPE = 0;
uart_intf.SECOND_STOP_BIT = 0;
uart_intf.DATA_BITS = 8;
```

```
    uvm_config_db#(int)::set(uvm_root::get(), "uart", "baud_rate",
    uart_intf.BAUD_RATE);

    uvm_config_db#(bit)::set(uvm_root::get(), "uart", "parity_enable",
    uart_intf.PARITY_ENABLE);

    uvm_config_db#(bit)::set(uvm_root::get(), "uart", "parity_type",
    uart_intf.PARITY_TYPE);

    uvm_config_db#(bit)::set(uvm_root::get(), "uart", "second_stop_bit",
    uart_intf.SECOND_STOP_BIT);

    uvm_config_db#(int)::set(uvm_root::get(), "uart", "data_bits",
    uart_intf.DATA_BITS);
```

```
enable_clock(10ns);
```

```
begin
```

```
uart_en_apb_seq my_seq;
```

```
my_seq = uart_en_apb_seq::type_id::create("my_seq");
```

```
my_seq.start(env.apb.seqr);
apb_intf.wait_till_idle();
```

```
end
```

```
phase.drop_objection(this);
```

```
endtask
```

Read this in slow motion.

First, the test raises an objection

That tells UVM not to let the phase end while configuration is still running.

Second, UART defaults are established

The UART interface is given known default values for baud rate, parity, stop bits, and data width.

Third, those values are also published through `uvm_config_db`

This allows other parts of the environment, such as monitors or scoreboards, to use the same assumptions.

Fourth, the clock is enabled

A design cannot meaningfully process transactions without a stable running clock.

Fifth, an APB setup sequence is executed

This programs the DUT through the same APB path a real system would use.

Finally, the objection is dropped

That tells UVM the configuration work for this phase is finished.

3.5 What an Objection Really Means

Objections confuse many beginners because the name sounds more abstract than the actual idea.

An objection is simply UVM's way of saying:

"I am still doing important work. Do not let this phase end yet."

The pattern is:

- raise an objection before starting meaningful work,
- do the work,
- drop the objection when the work is complete.

If all objections for that phase are dropped, UVM is free to move forward.

Why does this matter?

Because without objections, a phase could end too early:

- before a sequence finishes,
- before UART traffic has drained,
- before the scoreboard has observed the final transactions.

That would make the testbench fragile and misleading.

3.6 Why Derived Tests Should Stay Small

A trainee engineer sometimes feels that a test is not "real" unless it contains a lot of code. In good UVM, smaller tests are often better.

A derived test should usually focus on:

- what unique stimulus it starts,
- what traffic it wants to exercise,
- and when it should wait for interfaces to go idle.

Everything that is common should remain in the base test.

That is not just a style preference. It keeps the environment consistent and easier to debug.

3.7 The Role of Reset in Verification Quality

Reset is more than a ritual. It is one of the most important parts of making a test reproducible.

A clean reset sequence ensures:

- the DUT begins in a known state,
- stale interface values do not leak from previous activity,
- tests behave consistently from run to run.

One of the fastest ways to create hard-to-debug failures is to start stimulus before reset and configuration are complete.

3.8 The Main Lesson of This Chapter

If you leave this chapter understanding the sentence below, you are ready to move on:

"Phases keep the testbench organized, objections prevent phases from ending too early, and the base test centralizes all common setup so scenario tests stay focused."

That is the execution backbone of the environment.

Chapter 4: Agents, Sequences, and End-to-End Data Paths

What You Should Learn in This Chapter

This chapter explains what is inside an agent and then walks slowly through the most important TX and RX data paths in the testbench.

By the end, you should understand:

- what sequencers, drivers, and monitors each do,
- why APB and UART use separate agents,
- how `basic_write_test` proves the transmit path,
- how `basic_read_test` proves the receive path.

4.1 What Is Inside an Agent?

An agent is a container for everything needed to work with one interface.

In this example, each agent contains:

- a sequencer,
- a driver,
- a monitor,
- and an analysis port.

That sounds abstract at first, so let us slow it down.

The sequence item

A sequence item represents one transaction.

Examples:

- one APB write to `TX_DATA`,
- one APB read from `RX_DATA`,
- one UART byte being transmitted or received.

The sequence

A sequence creates one or more sequence items.

This is where test intent lives.

Examples:

- generate 256 random APB writes to the transmit data register,
- generate 256 random UART receive bytes.

The sequence describes what to do, not how to toggle individual pins.

The sequencer

The sequencer is the coordinator between the sequence and the driver.

A beginner does not need to overcomplicate this. Think of it as the delivery point that hands transactions from the sequence into the driver.

The driver

The driver converts a transaction into real signal activity.

For APB, that means driving the APB protocol correctly.

For UART, that means serializing bits with the correct timing and framing.

The monitor

The monitor observes interface activity and reconstructs transactions from what actually happened.

This distinction is essential:

- drivers create interface activity,
- monitors report interface activity.

A monitor should never "pretend" the DUT was correct. It must report what it truly saw.

4.2 Why APB and UART Use Different Agents

A trainee engineer may ask, "Why not have one big agent for the entire DUT?"

Because the APB and UART interfaces are doing fundamentally different jobs.

The APB agent

The APB agent is responsible for the software-visible side of the DUT.

It handles:

- register reads and writes,
- configuration programming,
- reading and writing data registers,
- observing APB responses.

The UART agent

The UART agent is responsible for the serial communication side.

It handles:

- injecting receive traffic,
- observing transmit traffic,
- decoding or encoding byte-level UART activity,
- tracking framing-related behavior.

Keeping these separate makes the environment easier to understand and easier to extend.

4.3 The Basic Write Test: APB to UART

The most important beginner transmit question is:

If software writes bytes through APB, do those same bytes come out of the UART transmitter?

That is the purpose of `basic_write_test`.

A representative stimulus setup looks like this:

```
random_apb_wdata_seq my_seq;
```

```
uvm_config_db#(int)::set(uvm_root::get(), "parameter",
```

```
"RANDOM_APB_WDATA_SEQ_LENGTH", 256);

my_seq = random_apb_wdata_seq::type_id::create("my_seq");
```

```
my_seq.start(env.apb.seqr);
```

This is a very common UVM pattern:

- configure the sequence behavior,
- create the sequence object,
- start it on the appropriate sequencer.

Step-by-step transmit flow

Step 1: The test starts an APB write sequence

The sequence is configured to generate repeated writes to TX_DATA at address 0x14.

Step 2: The APB sequence creates transactions

Conceptually, each transaction says:

- this is a write,
- the address is 0x14,
- the data is one randomized byte.

Step 3: The APB driver turns those transactions into bus activity

The driver performs the APB protocol correctly by driving the proper setup and enable timing on the APB interface.

Step 4: The DUT accepts the bytes into the transmit path

Once the bus write is accepted, the DUT pushes the data into its transmit path.

Step 5: The UART side begins serial transmission

The data is serialized and appears as UART activity on the serial interface.

Step 6: The UART monitor reconstructs the byte

The UART monitor watches the serial line, decodes the frame, and publishes a byte-level transaction.

Step 7: The scoreboard compares the APB write against the UART transmit byte

If they match, the transmit path is behaving correctly.

If they do not match, the scoreboard flags a failure.

That is a complete end-to-end check from a register write to a serial-line observation.

4.4 The Basic Read Test: UART to APB

The opposite direction is equally important.

The verification question is:

If serial data arrives on the UART receive side, can software later read that same data through APB?

That is the purpose of `basic_read_test`.

Step-by-step receive flow

Step 1: The test starts a UART receive sequence

The UART-side sequence generates randomized receive bytes.

Step 2: The UART driver serializes and drives the receive traffic

The bytes are driven into the DUT using real UART framing behavior.

Step 3: The DUT receives and stores the data

The DUT processes the serial input and makes the data available on the receive side, typically through an RX FIFO and receive data register path.

Step 4: An APB read sequence reads from `RX_DATA`

After receive traffic has been injected, the APB side reads from `0x18`.

Step 5: The APB monitor reports what data came back

The APB monitor observes the returned read data exactly as it appeared on the bus.

Step 6: The scoreboard compares the APB read against the UART receive byte

If the APB read value matches the UART receive queue, the receive path is working correctly.

That is the second major end-to-end proof in the environment.

4.5 Why Waiting for Idle Matters

Tests in this environment often wait for APB and UART interfaces to become idle after starting stimulus. That is not cosmetic.

It matters because:

- APB traffic may still be completing,
- UART serial transfers take longer than bus writes,
- monitors and scoreboards may still be receiving final observations.

If a phase ends too early, the test can appear to fail randomly or miss valid comparisons.

This is one of the most common sources of beginner confusion in protocol-based verification.

4.6 What You Should Be Able to Explain Now

Before moving on, you should be able to describe the following in your own words:

- what makes a sequence different from a driver,
- what makes a driver different from a monitor,
- why APB and UART are handled by different agents,
- and how one byte moves from APB write to UART observation, or from UART injection to APB readback.

If you can explain those flows clearly, you are already thinking like a verification engineer rather than just reading code mechanically.

Chapter 5: Scoreboard, Coverage, and What the Tests Prove

What You Should Learn in This Chapter

This chapter explains how the environment decides whether the DUT behaved correctly and how it measures what has been exercised.

By the end, you should understand:

- why the scoreboard is the main checker,
- why separate queues are used for APB and UART observations,
- how end-to-end compares are performed,
- and why coverage still matters even in a small environment.

5.1 Why the Scoreboard Is So Important

A UVM environment is not complete just because it can generate traffic. Real verification begins when the environment can decide whether the DUT was correct.

That is the scoreboard's job.

In this APB-UART example, the scoreboard is particularly educational because it compares two different views of the same design:

- what APB activity asked the DUT to do,
- what UART behavior actually happened.

This makes it much stronger than a checker that only looks at one interface in isolation.

5.2 The Scoreboard's Three Main Jobs

The scoreboard performs three important functions.

Job 1: Store APB and UART observations separately

The scoreboard keeps separate storage for:

- APB response items,
- UART transmit bytes,
- UART receive bytes.

Why separate queues?

Because APB and UART do not operate on the same time scale.

- APB transactions complete in bus cycles.
- UART transactions unfold at serial bit timing.

The scoreboard uses queues so each side can arrive when it is ready and still be compared correctly.

Job 2: Track active configuration

The scoreboard watches APB writes to configuration-related addresses.

For example:

- a write to the divider register changes baud-rate expectations,
- a write to the configuration register changes parity and stop-bit expectations.

This keeps the checking logic aligned with how the DUT was actually programmed.

Job 3: Compare the two important data flows

The scoreboard compares:

- APB writes to TX_DATA against observed UART transmit bytes,
- APB reads from RX_DATA against observed UART receive bytes.

That is the central end-to-end proof of this environment.

5.3 A Simplified View of the Compare Logic

A simplified version of the main scoreboard logic looks like this:

```
if (apb_item.pwrite == 1 && apb_item.paddr == 'h14) begin
    byte data;
```

```
wait (uart_tx_q.size());
```

```
    data = uart_tx_q.pop_front();

    if (data != apb_item.pwdata[7:0]) begin
        `uvm_error(get_type_name(), "TX data mismatch")
    end
end

else if (apb_item.pwrite == 0 && apb_item.paddr == 'h18) begin
    byte data;
```

```
wait (uart_rx_q.size());
```

```
    data = uart_rx_q.pop_front();

    if (data != apb_item.prdata[7:0]) begin
        `uvm_error(get_type_name(), "RX data mismatch")
    end
end
```

A trainee engineer should notice something very important here:

The scoreboard does not always compare two events instantly. Sometimes it stores one observation, waits for the matching observation to appear later, and then performs the comparison.

That is a very common real-world verification pattern.

5.4 What the Scoreboard Is Really Proving

Let us translate the scoreboard checks into plain language.

APB write to TX_DATA

When software writes a byte into the transmit register:

- the DUT should eventually send that byte through the UART transmit path,
- the UART monitor should observe that byte,
- the scoreboard should confirm the transmitted byte matches what software wrote.

APB read from RX_DATA

When the DUT has received a UART byte:

- the byte should become available through the receive data register path,
- an APB read should return that value,
- the scoreboard should confirm the APB read matches what the UART side received.

This is why the environment is educational: it proves agreement across interfaces.

5.5 Why Coverage Still Matters

A beginner sometimes thinks that if the scoreboard reports no errors, the environment is finished.

That is not true.

Pass fail checking answers:

- Did the observed checks succeed?

Coverage answers:

- What important scenarios have actually been exercised?

In this environment, functional coverage is collected for areas such as:

- APB transaction behavior,
- UART transaction behavior,
- register access behavior,
- test pass or fail status.

Coverage matters because you can easily have a test that never fails but still misses large parts of the behavior space.

Examples:

- you may never enable parity,
- you may never exercise some register access combinations,
- you may only test one data direction,
- you may never stress boundary conditions.

Coverage helps expose those blind spots.

5.6 What Each Core Test Proves

The environment does not need many tests if each test has a clear purpose.

base_test

This proves:

- the environment builds correctly,
- reset and configuration flow are working,
- the UVM phase structure is alive and coherent.

basic_write_test

This proves:

- APB-side transmit writes are accepted,
- the UART transmit path emits the expected bytes,
- the APB-to-UART end-to-end path is correct.

basic_read_test

This proves:

- UART receive traffic can be injected,
- the received data becomes visible through APB,
- the UART-to-APB end-to-end path is correct.

all_reg_access_test

This proves more about:

- register map accessibility,
- broader access-pattern coverage,
- exercising more than just the main data registers.

A trainee engineer should learn from this that a strong test suite usually mixes:

- core data-path tests,
- control and configuration tests,
- coverage-broadening tests.

5.7 What the Scoreboard Does Not Automatically Prove

Another useful lesson is to understand the limits of the current checking model.

The scoreboard is very strong for:

- TX data comparisons,
- RX data comparisons,
- basic APB and UART observation correlation,
- configuration-aware checking.

But it does not automatically prove every possible feature unless you extend it.

Examples that may require extra checking logic include:

- interrupt behavior,
- FIFO threshold behavior,
- parity error reporting,
- illegal access behavior,
- corner cases around flush operations.

This is an important professional habit: always ask not only what is checked, but also what is not yet checked.

5.8 The Main Lesson of This Chapter

If you leave this chapter understanding the sentence below, you are ready for the final extension chapter:

"The scoreboard turns observations into proof of correctness, and coverage tells us how much of the intended behavior space we have actually exercised."

That is the quality-assurance core of the environment.

Chapter 6: Extending the Environment as a New Verification Engineer

What You Should Learn in This Chapter

This final chapter focuses on practical growth.

By the end, you should understand:

- how to add a new test without damaging the structure,
- when to reuse existing sequences,
- when the scoreboard must change,
- and what beginner mistakes to avoid.

6.1 A Safe Way to Think About Extensions

When a trainee engineer first adds a new test, the most common mistake is to change too many parts of the environment at once.

A safer approach is:

- decide what behavior you want to verify,
- identify whether the environment already has the right stimulus path,
- add the smallest new piece needed,
- keep common setup in the base test,
- extend the scoreboard only if the new behavior introduces a new kind of check.

This keeps the environment stable while you learn.

6.2 Reuse Before You Create

Before writing a new sequence or a new checker, ask two simple questions.

Question 1: Do I already have the right traffic type?

If your new test still uses:

- APB register reads and writes,
- UART receive traffic,
- UART transmit observations,

then you may be able to reuse existing sequences or extend them slightly rather than starting from zero.

Question 2: Is the scoreboard already comparing the behavior I care about?

If your new test is still fundamentally about:

- APB writes matching UART transmit bytes,
- UART receive bytes matching APB read data,

then the current scoreboard may already be sufficient.

That is the first thing to check before adding more infrastructure.

6.3 A Minimal Derived Test Template

A good beginner template for a new test looks like this:

```
class my_test extends base_test;

    `uvm_component_utils(my_test)

    function new(string name = "my_test", uvm_component parent = null);
```

```
super.new(name, parent);
```

```
endfunction

task main_phase(uvm_phase phase);
```

```
phase.raise_objection(this);
// Start APB and or UART sequences here.
apb_intf.wait_till_idle();
uart_intf.wait_till_idle();
phase.drop_objection(this);
```

```
endtask

endclass
```

The key lesson here is what is not present:

- no duplicated reset code,
- no duplicated clock startup,
- no duplicated base configuration.

That setup already belongs to base_test.

6.4 When the Scoreboard Needs to Grow

A trainee engineer should be careful not to assume the scoreboard magically understands every new feature.

If your new test targets behavior such as:

- interrupts,
- FIFO almost-full thresholds,
- parity error signaling,
- invalid access response behavior,
- flush corner cases,

then the scoreboard may need new compare logic or new state tracking.

A practical rule is this:

If the test is checking something the scoreboard never observes or compares today, the scoreboard probably needs an update.

6.5 When Coverage Needs to Grow

The same idea applies to coverage.

If you introduce a new important feature or mode, ask:

- do I have coverage that reflects this feature,
- or am I only relying on pass fail checking?

For example, if you add parity-enabled testing, you may also want coverage points that record whether parity-enabled traffic has really been exercised.

Coverage is how you make sure the new feature is not only supported in code, but also intentionally explored by tests.

6.6 Common Beginner Mistakes

This section is especially useful for trainees because these mistakes are very common.

Mistake 1: Putting scenario logic in the driver

The driver should execute transactions, not invent them.

Scenario intent belongs in tests and sequences.

Mistake 2: Putting checking inside the monitor

The monitor should observe and report.

The scoreboard should judge correctness.

Mistake 3: Repeating setup in every test

Reset, clock startup, and common configuration belong in the base test.

Mistake 4: Ending a phase too early

If you do not raise and drop objections correctly, or if you do not wait for APB and UART activity to drain, the environment may miss valid late-arriving observations.

Mistake 5: Assuming no errors means enough testing

A test can pass while still covering only a small part of the intended behavior space.

That is why coverage matters.

6.7 A Practical Checklist for Writing a New Test

When you add a test, use this checklist.

- What exact DUT behavior am I trying to prove?
- Is this behavior visible on APB, UART, or both?

- Which existing sequence gets me closest to the needed stimulus?
- Do I need a new sequence or only a new sequence length or constraint?
- Does the existing scoreboard already compare this behavior?
- Do I need new coverage for this feature?
- Have I waited long enough for all activity to complete before ending the phase?

If you can answer these questions clearly, your test will usually be much easier to debug.

6.8 Final Takeaway for a New Verification Engineer

At first, UVM can feel like a large framework with many moving parts. But this APB-UART environment shows that the core idea is actually straightforward.

- Tests express intent.
- Sequences generate transactions.
- Drivers create signal activity.
- Monitors observe reality.
- Scoreboards compare intent and observation.
- Coverage tells you what parts of the behavior space were exercised.

If you can hold that model in your head, you already understand the foundation of UVM.

Everything more advanced is usually an extension of those same ideas.